

Raport Sprintu 5

Projekt: SpotkajMnie

Zespół: Filip Sobieraj, Paweł Wdowik, Karolina Kolarz

15 czerwca 2026

1 Implementacja Warstwy Backendowej

Główne elementy zrealizowane w kodzie:

- **Model Domenowy:** Zaprojektowano struktury danych odzwierciedlające wymagania funkcjonalne, w tym encje `User`, `TimeSlot` oraz `Event`. Zastosowano `std::shared_ptr` do bezpiecznego zarządzania pamięcią i powiązaniem między obiektami.
- **Serwis Logiki Biznesowej:** Klasa `EventSchedulingService`, gwarantuje szybkie działanie algorytmu wyliczającego optymalny termin spotkania. Klasa ta analizuje oddane głosy i automatycznie wskazuje termin o najwyższej frekwencji.
- **Interfejs Powiadomień:** Stworzono interfejs abstrakcyjny `NotificationService`, który pozwala na elastyczne podpinanie różnego rodzaju alertów dla użytkowników, oddzielając logikę decyzyjną od warstwy prezentacji.
- **Aplikacja Konsolowa (CLI):** W pliku `main.cpp` przygotowano interaktywne środowisko testowe. Umożliwia ono symulację pełnego procesu z perspektywy różnych ról: logowanie użytkowników, tworzenie wydarzeń, proponowanie przedziałów czasowych przez organizatora, oddawanie głosów na pasujące terminy oraz finalne uruchomienie algorytmu rozstrzygającego. Służy do testowania ręcznej aplikacji.

2 Testy Jednostkowe

Testy jednostkowe napisano przy użyciu biblioteki **Google Test (GTest)**, zintegrowanej bezpośrednio z plikiem konfiguracyjnym `CMake`.

Do weryfikacji powiadomień wykorzystano wzorzec `Mock` (`MockNotificationService`), co pozwoliło na śledzenie zachowania algorytmu bez konieczności uruchamiania pełnego interfejsu. Zaimplementowano zestaw 7 testów:

- **Idealne dopasowanie:** Potwierdzono, że system automatycznie wybiera optymalny termin spełniający kworum i przygotowuje powiadomienie o ustaleniu daty.
- **Obsługa remisu:** Zweryfikowano scenariusz, w którym dwa terminy mają dokładnie taką samą liczbę głosów. Zgodnie z założeniami, system nie podejmuje decyzji samodzielnie, lecz prosi organizatora o ostateczny wybór.
- **Brak kworum:** Przetestowano sytuację, w której optymalny termin zebrał niewystarczającą liczbę uczestników w stosunku do minimalnego progu. System poprawnie wstrzymuje zatwierdzenie i oznacza wydarzenie statusem „Wymaga uwagi”.

- **Przypadki brzegowe i odporność:** Dodatkowe testy potwierdziły prawidłowe zachowanie systemu w sytuacjach skrajnych – m.in. zignorowanie obliczeń w przypadku braku zaproponowanych terminów, odrzucenie wyboru w przypadku zera oddanych głosów, poprawne zatwierdzenie przy spełnieniu kworum „na styk” oraz niezależność wyliczeń od kolejności dodawania terminów.